

brain is loaded to the kernel, which eventually generates the artificial intelligence response.

Coding and testing the program

The aim is to build an automated response generator which will respond through XMPP. First, download PyAIML and install it in your system. PyAIML provides proper back-end to respond to chat inputs. For installing PyAIML, just untar the package and type "python setup.py install" on the shell prompt. Now download and install XMPPPY package. For this, untar the package and run Python. You can download PyAIML from <http://pyaiml.sourceforge.net> and XMPPPY from <http://xmpppy.sourceforge.net>

To even install the above software, create a simple shell script in any text editor as given below and save it as setup.sh:

```
tar xvf pyaiml.tar.gz
tar xvf xmpppy0.5.0rc1.tar.gz
cd pyaiml
sudo python setup.py install
cd ..
cd xmpppy0.5.0rc1
sudo python setup.py install
cd ..
rm -f pyaiml
rm -f xmpppy0.5.0rc1
```

Now run the following command on shell prompt to install PyAIML and XMPPPY:

```
sudo ./setup.sh
```

This should complete without any error. Should you get any error message, resolve that before proceeding further.

Let's now come to the main part of the project. Create a Python program named susie.py as below:

```
susie.py
import xmpp
import time
import aiml
import sys
def load_brain(): # to load brain if
available, else generate it from the
available libraries
    kern = aiml.Kernel()
```

```
    try:
        kern.bootstrap(brainFile
= "standard.brn")
        brainLoaded = True
    except:
        kern.bootstrap(learnFiles
+"stdstartup.xml", commands="load
aiml b")
        brainLoaded = True
        kern.saveBrain("standard.
.brn")
    return(kern)
def messageCB(sess,mess): # to get
incoming messages from xmpp,generate
reply from kernel and send it back
    nick=mess.getFrom( )
    getResource( )
    text=mess.getBody( )
    reply=kern.respond(text)
    sess.send(xmpp.Message(mess.
getFrom(),reply))
def LoopyFn(conn): # to sustain the
connection
    try:
        conn.Process(1)
    except KeyboardInterrupt:
        return 0
    return 1
def main_process(): #create the
connection, log in
    login = sys.argv[1] #if it is
xyz@gmail.com just put xyz
    pwd = sys.argv[2]
    connection = xmpp.Client('gmail.
com') #different for different xmpp
clients
    connection.connect(server=
('talk.google.com',5222))
    connection.RegisterHandler
('message',messageCB)
    connection.auth(login,pwd,
"Bot",saal=0)
    connection.sendInitPresence()
    while 1:
        connection.Process(1)
        pass
if __name__ == '__main__':
    kern=load_brain( )
    main_process( )
```

Before running this program, you need to know a little about loading and saving the brain file which will be used by the susie.py.

Loading and saving brains. If

your bot's AIML program isn't giving appropriate response, you can significantly reduce the start-up time by using the Kernel's loadBrain() and saveBrain() methods. These functions let you dump the contents of your bot's 'brain' to a file on the disk. The next time your bot runs, rather than re-parsing all the AIML files, it can just load its brain from the last session and pick up right from where it left off!

For this, you need to create the brain dump file. First, create a Kernel object and load the standard AIML set. Remember that just loading 'std-startup.xml' isn't enough—the bulk of the AIML files aren't loaded until you pass 'load aiml b' to the Kernel's respond() method.

Create the brain dump file as follows:

```
import aiml
k = aiml.Kernel()
k.learn("std-startup.xml")
k.respond("load aiml b")
k.saveBrain("standard.brn")
```

Save the above commands in a text file called create_brain.py. Run the create_brain.py script on the shell prompt, in the folder where rest of the files are stored along with the directory named 'standard.' This will create standard.brn file and save it in the working directory.

You can download all the above source codes from the link: <https://bitbucket.org/atmb4u/susie>

Now check the program for proper working by running the following command:

```
python susie.py <username> <password>
For example, if your gmail ID is
google@gmail.com, the command will be:
```

```
python susie.py google 123456
```

The program lacks simultaneous multiple chats. However, its standard responses are satisfactory.

EFY note. The source code of this article has been included in this month's EFY-CD and is also available on www.efymag.com website. ●

The author, a B.Tech in computer science and engineering, is a Python and Open Source enthusiast doing projects on AI programming